

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Desenvolvimento de Serviços e APIs

Desenvolvendo uma API RESTful em Camadas com Node.js e Express

Arquitetura em Camadas (Rotas, Controllers, Services)

Separação de responsabilidades para APIs escaláveis



Professor: Carlos Bruno Oliveira Lopes

Índice da Aula



O que é API e REST



Por que Arquitetura em Camadas



Estrutura de pastas do projeto



Middleware express.json



Hello World (antes e depois)



Testes com Insomnia/Postman



Ciclo Request → Response



Camadas: Router, Controller, Service



Configuração do ambiente



Implementação das camadas



Parâmetros de requisição



Boas práticas e próximos passos

Objetivos da Aula

-  Compreender o que é uma API e o estilo arquitetural REST
-  Entender o ciclo Request/Response
-  Compreender a Separação de Responsabilidades (SoC)
-  Estruturar projeto em camadas: Rotas, Controllers e Services
-  Criar e refatorar rotas (Hello World, CRUD básico)
-  Lidar com Params, Query e Body nas requisições

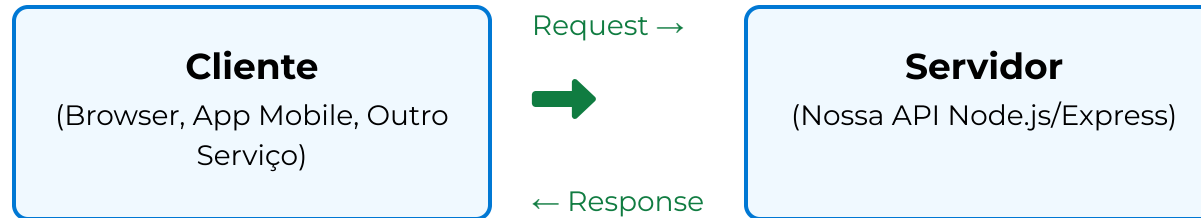
Arquitetura em Camadas (Rotas, Controllers, Services)

O que é uma API? E o que é REST?

-  **API: Interface de Programação de Aplicações**
Permite comunicação entre sistemas via contratos (endpoints, formatos)
-  **REST: Representational State Transfer**
Estilo arquitetural que utiliza HTTP, recursos (URLs), verbos (GET/POST/PUT/DELETE), e representações (JSON)
-  **Características**
Stateless, cacheável, interface uniforme, arquitetura em camadas
-  **Benefícios**
Simplicidade, escalabilidade, interoperabilidade, desacoplamento

Arquitetura em Camadas (Rotas, Controllers, Services)

Ciclo de Requisição e Resposta (HTTP)



Cliente envia **Request**:

- Método (GET, POST, PUT, DELETE...)
- URL (/api/users, /products/1...)
- Headers (Authorization, Content-Type...)
- Body - opcional (dados em JSON, form-data...)



Servidor retorna **Response**:

- Status code (200, 201, 404, 500...)
- Headers (Content-Type, Cache-Control...)
- Body - JSON, HTML, etc. (dados solicitados)



No Express.js:

- req - Objeto de requisição (entrada)
- res - Objeto de resposta (saída)
- Exemplo: `app.get('/users', (req, res) => { ... })`

O fluxo HTTP é stateless: cada requisição é independente

Por que Arquitetura em Camadas?

-  Manutenibilidade: menos acoplamento, código mais legível e organizado
-  Escalabilidade: camadas independentes facilitam evolução do sistema
-  Testabilidade: serviços testáveis sem depender de HTTP
-  Reuso: lógica de negócio reaproveitável em outros canais (CLI, jobs)
-  Organização: responsabilidades claras e bem definidas

Separação de Responsabilidades (Separation of Concerns)

Camadas: Papéis e Responsabilidades



Router (Rotas):

Define endpoints e métodos HTTP; encaminha a requisição para o Controller apropriado. É a porta de entrada da API.



Controller:

Orquestra a requisição; extrai dados de req; chama o Service; formata e envia a resposta JSON. Conhece HTTP mas não implementa regras de negócio.



Service:

Contém a lógica de negócio pura; não conhece req/res; executa operações no banco de dados; fornece resultados para o Controller.



A separação em camadas permite código mais organizado, testável, reaproveitável e facilita a manutenção do sistema ao longo do tempo.

Arquitetura em Camadas (Rotas, Controllers, Services)

Estrutura de Pastas do Projeto

```

  /src
    /routes
      index.js
      user.routes.js
    /controllers
      user.controller.js
    /services
      user.service.js
  index.js
  package.json
```

Organização por camadas: separação de responsabilidades

Configuração do Ambiente e Dependências



Requisitos: Node.js LTS instalado (<https://nodejs.org>)



Iniciar projeto:

```
npm init -y
```



Instalar Express:

```
npm i express
```



Instalar Nodemon (dev):

```
npm i -D nodemon
```



Script de desenvolvimento no package.json:

```
"scripts": {  
  "dev": "nodemon src/index.js"  
}
```

Node.js + Express + Nodemon = Ambiente de desenvolvimento completo

Arquivo Principal: src/index.js

```
const express = require('express');
const userRoutes = require('./routes/user.routes');
const app = express();
const PORT = 3000;
// Middleware para ler JSON
app.use(express.json());
// Prefixo opcional para as rotas
app.use('/api', userRoutes);
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

Ponto de entrada da aplicação: configura middleware, rotas e inicia o servidor

Service: src/services/user.service.js

```
// Simulando um banco de dados em memória
const users = [{ id: 1, name: 'João' }];

class UserService {
  getAll(limit) {
    return typeof limit === 'number' ? users.slice(0, limit) : users;
  }

  create(userData) {
    const newUser = { id: users.length + 1, ...userData };
    users.push(newUser);
    return newUser;
  }

  getById(id) {
    return users.find(u => u.id === parseInt(id));
  }
}

module.exports = new UserService();
```

Camada Service: contém apenas a lógica de negócio e não conhece o protocolo HTTP

Controller: src/controllers/user.controller.js

```
const userService = require(' ../services/user.service');
class UserController {
  getAll(req, res) {
    const limit = req.query.limit ? parseInt(req.query.limit) : undefined;
    const users = userService.getAll(limit);
    res.json({ success: true, data: users });
  }
  create(req, res) {
    const newUser = userService.create(req.body);
    res.status(201).json({ success: true, data: newUser });
  }
  getById(req, res) {
    const user = userService.getById(req.params.id);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'Usuário não encontrado'
      });
    }
    res.json({ success: true, data: user });
  }
}
module.exports = new UserController();
```

Responsabilidade do Controller: Orquestrar a requisição, extrair dados, chamar o Service e responder ao cliente

Routes: src/routes/user.routes.js

```
const { Router } = require('express');
const userController = require(' ../controllers/user.controller');
const router = Router();
// Definição das rotas de usuário
router.get('/users', userController.getAll);
router.post('/users', userController.create);
router.get('/users/:id', userController.getById);
module.exports = router;
```

Camada de Rotas: define os endpoints (URLs) e verbos HTTP, direcionando para os Controllers apropriados

Registrando Rotas e Prefixos



Em `src/index.js` utilizamos `app.use('/api', userRoutes)`

```
// index.js
const userRoutes = require('./routes/user.routes');
app.use('/api', userRoutes);
```



Benefícios do prefixo:

- Versionamento da API (ex.: `/api/v1/users`, `/api/v2/users`)
- Organização por domínio ou contexto (`/api/auth`, `/api/admin`)
- Separação de APIs públicas e privadas (`/public`, `/admin`)



Dica: use um arquivo `src/routes/index.js` para centralizar routers

```
// routes/index.js
const userRoutes = require('./user.routes');
const authRoutes = require('./auth.routes');
module.exports = (app) => {
  app.use('/api/users', userRoutes);
  app.use('/api/auth', authRoutes);
};
```



Parâmetros da função `app.use()`: **(path, router)**, onde `path` é opcional e pode ser uma string ou array de strings

Hello World: Antes e Depois (Refatoração)

</> Antes (tudo no index.js)

```
const express = require('express');
const app = express();
const PORT = 3000;

// Rota Hello World
app.get('/hello', (req, res) => {
  res.send('Hello World');
});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

⌵ Depois (com camadas)

```
// routes/hello.routes.js
const { Router } = require('express');
const router = Router();

router.get('/hello', (req, res) => {
  res.json({ message: 'Hello World' });
});

module.exports = router;

// index.js
const helloRoutes = require('./routes/hello.routes');
app.use('/api', helloRoutes);
```

Vantagens: Respostas em JSON padronizadas, organização por responsabilidade, prefixo unificado (/api)

Middleware express.json()



Objetivo

Permite ler e processar o corpo da requisição em formato JSON, transformando-o em um objeto JavaScript acessível via `req.body`



Como usar

Coloque antes das rotas em seu arquivo principal:

```
// Aplica o middleware em todas as rotas
app.use(express.json());
```



Sem o middleware

O `req.body` virá como `undefined` em requests com JSON, gerando erros ao tentar acessar propriedades



Alternativas e extensões

Combine com bibliotecas de validação como Zod/Joi para validar esquemas após o parse do JSON

Express middleware para processamento de JSON

Parâmetros de Requisição: Visão Geral



Params

Parte da URL depois de ":" `/users/:id`

Acessados via `req.params` (ex: `req.params.id`)



Query

Após o "?" na URL `/users?limit=10&sort=name`

Acessados via `req.query` (ex: `req.query.limit`)



Body

Dados enviados no corpo da requisição (POST, PUT, PATCH)

Requer middleware `express.json()`

Acessados via `req.body` (ex: `req.body.name`)



Headers

Metadados da requisição (Content-Type, Authorization)

Acessados via `req.headers` (ex: `req.headers.authorization`)

Todos disponíveis através do objeto de requisição (req) no Express

Exemplo com Params (GET /users/:id)

```
// Router - Definindo rota com parâmetro na URL

const { Router } = require('express');
const userController = require('../controllers/user.controller');
const router = Router();

// O ":id" captura qualquer valor nesta posição da URL
router.get('/users/:id', userController.getById);

module.exports = router;

// Controller - Acessando o parâmetro da URL

getById(req, res) {
  // req.params contém todos os parâmetros da URL
  const userId = req.params.id;
  const user = userService.getById(userId);

  // Verificação se o usuário existe
  if (!user) {
    return res.status(404).json({
      success: false,
      message: 'Usuário não encontrado'
    });
  }
  res.json({ success: true, data: user });
}
```

Params - Valores extraídos diretamente da URL (ex: /users/1 → id=1)

Exemplo com Query (GET /users?limit=10)

 user.controller.js

```
class UserController {
  getAll(req, res) {
    // Extraindo o parâmetro limit da query string
    const limit = req.query.limit ? parseInt(req.query.limit) : undefined;
    const users = userService.getAll(limit);
    res.json({ success: true, data: users });
  }
}
```

 user.service.js

```
class UserService {
  getAll(limit) {
    // Retorna todos ou apenas um limite de usuários
    return typeof limit === 'number' ? users.slice(0, limit) : users;
  }
}
```

```
// URL da requisição: GET /api/users?limit=10
// Resposta: { "success": true, "data": [ ...primeiros 10 usuários] }
```

req.query permite acessar parâmetros opcionais da URL que vêm após o "?"

Exemplo com Body (POST /users)

```
// Middleware global (index.js)
const express = require('express');
const app = express();
// Middleware para ler JSON do corpo da requisição
app.use(express.json());

-----

// Rota (routes/user.routes.js)
const { Router } = require('express');
const userController = require(' ../controllers/user.controller');
const router = Router();
router.post('/users', userController.create);

-----

// Controller (controllers/user.controller.js)
create(req, res) {
  const newUser = userService.create(req.body); // Ex.: { "name": "Maria" }
  res.status(201).json({ success: true, data: newUser });
}
```

O Body vem do cliente como JSON e é parseado pelo middleware `express.json()`

Boas Práticas de Resposta



Consistência: sempre retorne JSON com a mesma estrutura

```
{ success: boolean, data?: any, message?: string, errors?: any }
```



Status codes corretos

200 (OK), 201 (Created), 400 (Bad Request), 404 (Not Found), 500 (Internal Error)



Mensagens claras e úteis ao cliente para facilitar o consumo da API



Não exponha detalhes sensíveis de erros internos (stack traces, queries SQL)

Arquitetura em Camadas (Rotas, Controllers, Services)

Testando com Insomnia/Postman



GET `http://localhost:3000/api/users`

Lista todos os usuários cadastrados



GET `http://localhost:3000/api/users/1`

Busca um usuário específico pelo ID



GET `http://localhost:3000/api/users?limit=1`

Utiliza parâmetros de consulta para filtrar resultados



POST `http://localhost:3000/api/users`

Body (JSON): `{ "name": "Maria" }`



Verifique status, headers e body de resposta

Status codes: 200 (OK), 201 (Created), 404 (Not Found)

Testando sua API com ferramentas especializadas para desenvolvedores

Próximos Passos e Evoluções

-  Validação de dados (Zod/Joi) no Controller
-  Middlewares: autenticação, logs, tratamento de erros
-  Centralização de erros (error handler)
-  Persistência real: bancos (PostgreSQL, MongoDB) e camada de repositório
-  Versionamento de API (v1, v2) e documentação (OpenAPI/Swagger)

Evoluções para uma API robusta e profissional

Encerramento



Resumo

- Você aprendeu o porquê e o como da Arquitetura em Camadas
- Implementou Service, Controller e Routes com Express
- Lidou com Params, Query e Body nas requisições



Atividade Sugerida

Exercícios práticos:

- Adicionar rotas PUT/PATCH e DELETE para users
- Implementar validação de entrada e tratamento de erros centralizado



Perguntas?

Estou à disposição para esclarecer suas dúvidas